

How My Pal JSON works

Deep analysis generated 2026-09-07 · app booted via docker compose; screenshots are real captures · model claude-sonnet-5

Overview

My Pal JSON is a single-page, self-hosted web tool for testing HTTP/JSON APIs and for quickly turning JSON payloads into strongly-typed code models. It is aimed at developers who want a lightweight, local alternative to tools like Postman for firing off GET/POST/PUT requests, inspecting the JSON schema of a response, and generating boilerplate model classes/structs in six languages (C#, Python, JavaScript, C++, Java, Go) without leaving the browser. The whole application ships as a small Flask app with a single HTML page and a set of vanilla JS modules, and is distributed as a Docker image or via docker-compose for zero-setup local use.

End to end, the user builds a request in the top bar (HTTP method, URL, headers with an "Authorization/Bearer" convenience toggle, and a JSON body in the left textarea), then clicks Send. The browser posts the assembled request description to the Flask backend's `/proxy_request` endpoint, which performs the actual outbound HTTP call server-side (avoiding CORS restrictions) using the `requests` library and returns the status code and body. The response is pretty-printed back into the same textarea, and the UI auto-switches to a "Response" tab. Independently of sending a live request, the user can paste/drop any JSON into the input box and either "Analyze" it (POST to `/analyze`, which walks the JSON with `anytree` and returns an ASCII tree of its shape) or "Model" it (POST to `/model?language=...`, which recursively walks the JSON and emits class/struct source code text in the selected language). Both results are shown in the right-hand output textarea, which also supports one-click copy.

The app also has lightweight "session management" features that live entirely client-side: a header grid with dynamic add/remove rows and an eye toggle to hide/show it, a request/response tab switcher backed by an in-memory `dataStore`, a light/dark theme toggle that swaps stylesheets, and a browser-local IndexedDB-backed history feature (Save/History buttons) that stores past API calls (URL, method, headers, body) so users can reload or delete them later without any server-side persistence.

The intended audience is individual developers or small teams doing quick, ad hoc API exploration and needing fast scaffolding of typed models from sample JSON responses — essentially a "Postman + JSON-to-code generator" mashup runnable in a single Docker container with no external dependencies besides the Python packages Flask, flask-cors, requests, and anytree.

Architecture

The backend is a Flask application (`web/app.py`) that registers three blueprints as the entire server-side surface: `analyze_bp` (`web/analyze.py`, route `/analyze`), `model_bp` (`web/model.py`, route `/model`), and `proxy_bp` (`web/proxy.py`, route `/proxy_request`). The root route `/` simply renders the single Jinja template `web/templates/json-analyzer.html`, which is the entire frontend shell; there is no separate SPA framework — the page loads a dozen small, single-purpose vanilla JS files from `web/static/scripts/`, several as ES modules (`send-request.js`, `tab-switch.js`, `headers-management.js`, `save-call.js`, `history-modal.js`) so they can import/export shared state like the `dataStore` object. Styling is handled by two hand-written CSS files (`style.css` for dark theme, `lightstyle.css` for light theme) swapped by `theme-toggle.js` via changing the `<link>` href. CORS is enabled globally via `flask_cors.CORS(app)`, and in dev mode (`FLASK_DEBUG` unset/non-production) the app auto-opens a browser tab via a `Timer`.

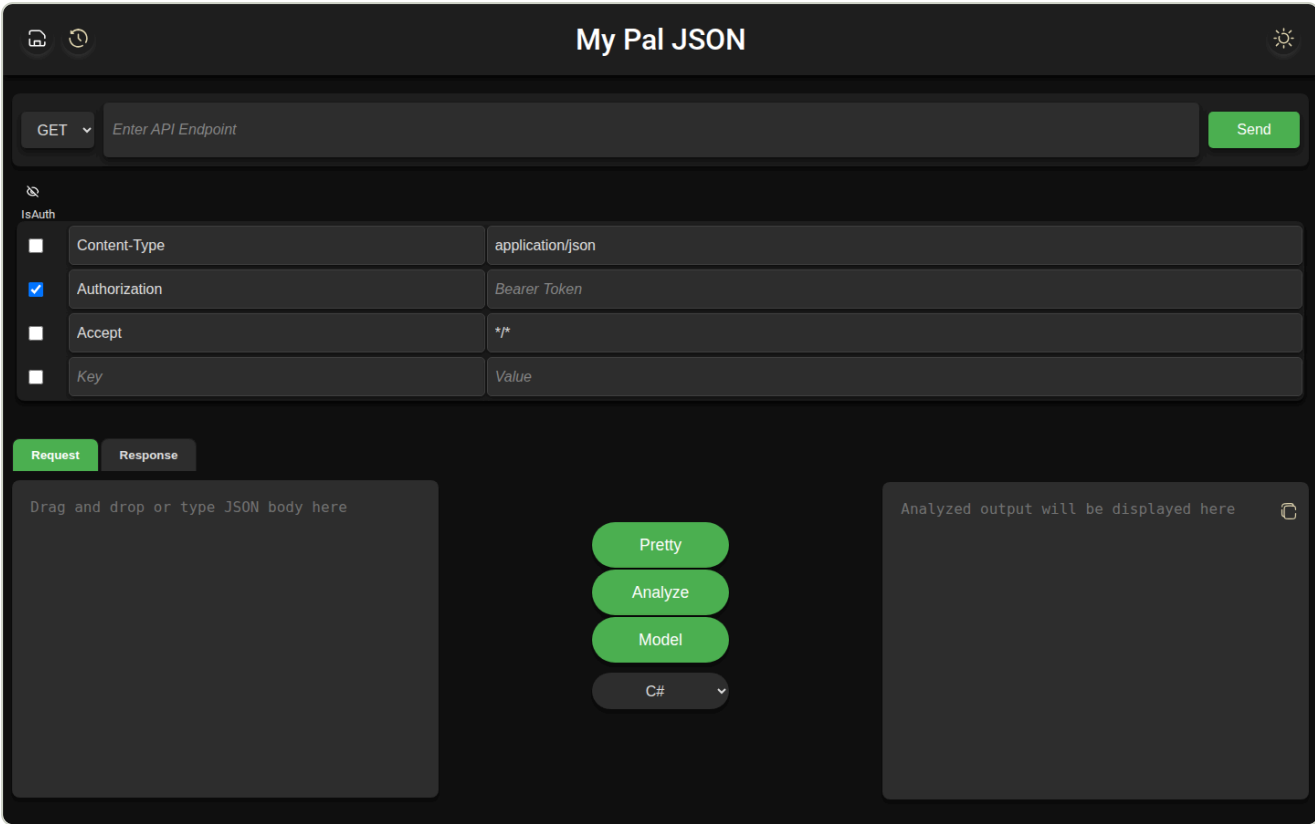
Data flow for the three core features is: (1) **Proxy/Send** — the browser never calls third-party APIs directly; `send-request.js` posts `{apiUrl, httpMethod, headers, body}` as JSON to Flask's `/proxy_request`, which uses Python's `requests.request(...)` to make the real call server-side and returns `{status_code, content}` as JSON, sidestepping browser CORS restrictions on arbitrary target APIs. (2) **Analyze** — `validate-json.js` posts the raw parsed JSON body to `/analyze`; `analyze.py` recursively builds an `anytree.Node` tree (collapsing lists to their first element, marking array keys with `[ ]`) and renders it with `RenderTree` into a plain-text tree string returned as `{tree: ...}`.

(3) **Model** — ``model-json.js`` posts JSON to ``/model?language=<lang>``; ``model.py`` picks a root class name/data (unwrapping a single-key dict, using ``Root`` for multi-key objects or arrays) and dispatches to one of six generator modules under ``web/generators/`` (``csharp.py``, ``python.py``, ``javascript.py``, ``cplusplus.py``, ``java.py``, ``go.py``), each of which recursively walks the JSON structure, infers a language-appropriate type/class/struct definition (including nested classes for nested objects/arrays-of-objects), and returns generated source as a single string in ``class_content``.

Client-side persistence (Save/History/Delete) is implemented purely in the browser using IndexedDB (``open-database.js`` opens a ``MyPalJsonDB`` database with an ``apiCalls`` object store), with ``save-call.js`` writing records (with duplicate detection ignoring auth headers), ``history-modal.js`` rendering saved calls into a modal and allowing "load" (repopulating the form via ``loadApiCallIntoForm``), and ``delete-record.js`` handling deletion with a confirm dialog — none of this touches the Flask backend. There is no database, authentication system, or multi-user concept on the server; the Flask backend is stateless and exists solely to proxy outbound HTTP calls and run the two JSON-transformation endpoints (``/analyze``, ``/model``).

Page: /

The single page of the application — a combined API request tester, JSON schema analyzer, and JSON-to-code model generator, served by Flask's `serve_html`` view rendering `json-analyzer.html``.



Walkthrough

A user lands on the page and sees a request bar at top (method dropdown, URL input, Send button), a headers grid below it (pre-populated with `Content-Type``, `Authorization``, and `Accept`` default rows plus one blank row for adding more), and a three-column workspace below that: a left panel with Request/Response tabs and a JSON textarea (supporting drag-and-drop of `.json`/text` files), a middle column of action buttons (Pretty, Analyze, Model) plus a language dropdown, and a right panel with an output textarea and copy button. The user types or drops JSON into the left textarea, optionally edits headers (checking the auth checkbox on a row applies a `Bearer `` prefix to that header's value), picks an HTTP method and enters an endpoint URL, then clicks Send to have the server proxy the live API call and display the JSON response (auto-switching to the Response tab). Alternatively, without sending anything, the user can click Pretty to reformat the textarea JSON locally, Analyze to see a tree-shaped schema in the output box, or select a language and click Model to generate class/struct source code for that JSON in the output box, which can then be copied with the copy button. The Save and History icon buttons in the header let the user persist the current request configuration to browser storage and later reload or delete it from a modal list, and the theme toggle switches between dark and light stylesheets.

Controls

CONTROL	WHAT IT DOES
Send (button#sendBtn)	Gathers method/URL/headers/body from the form, POSTs to Flask route <code>/proxy_request`</code> (proxy.py <code>proxy_request`</code>), which performs the real HTTP call via <code>requests.request`</code> and returns

status/content; response is JSON-stringified into ``inputjson`` and the Response tab is activated (send-request.js).

HTTP method select (select#httpMethod)	Sets the method (<code>`GET`/`POST`/`PUT`</code>) sent in the <code>`/proxy_request`</code> payload; also settable indirectly by loading a history record (history-modal.js <code>`loadApiCallIntoForm`</code>).
API endpoint input (input#apiUrl)	Holds the target URL passed as <code>`apiUrl`</code> to <code>`/proxy_request`</code> ; required non-empty for Save (save-call.js checks <code>`apiUrl.trim()`</code>).
Header auth checkbox (input.auth-checkbox, unlabeled)	When checked, marks a header row as an Authorization/Bearer token; on Send, its value is prefixed with <code>`Bearer `</code> (send-request.js), and on Save it's excluded from duplicate-detection comparison (save-call.js <code>`filterAuthHeaders`</code>).
Header Key input (input.header-key-input, placeholder "Key")	Names a custom header; typing in the last empty row auto-creates a new blank row and setting placeholder to "Bearer Token" if key is <code>`Authorization`</code> (headers-management.js <code>`createHeaderRow`/`addInputListeners`</code>).
Header Value input (input.header-value, placeholder "Value"/"Bearer Token")	Supplies the header's value, combined with the key when building the request headers object in send-request.js.
Toggle Headers button (button#toggleHeadersBtn)	Shows/hides the headers grid and the "IsAuth" label, swapping the eye/eye-slash icon (headers-toggle.js).
Save button (button#saveBtn)	Reads apiUrl/method/body/headers, checks IndexedDB for a duplicate via <code>`isDuplicateCall`</code> , and if unique, writes a new record to the <code>`apiCalls`</code> object store, showing a status message in the output box (save-call.js <code>`saveApiCall`</code>).
History button (button#historyBtn)	Opens the IndexedDB database and calls <code>`loadApiCallHistory`</code> to populate and display the History modal with all saved API calls (history-modal.js).

History modal Load button (button.load-button, per item)	Calls <code>loadApiCallIntoForm`</code> to repopulate the URL, method, body, and headers fields from a saved record and closes the modal (history-modal.js).
History modal Delete button (button.delete-button, per item)	Opens a confirm dialog; on confirmation, deletes the record from IndexedDB by id and reloads the history list (delete-record.js).
Confirm/Cancel delete (button#confirmDeleteBtn / #cancelDeleteBtn)	Executes or cancels the pending IndexedDB delete transaction triggered by the delete icon (delete-record.js).
Request tab (button#requestTab)	Switches the left textarea to show the stored request JSON (<code>dataStore.requestData`</code>) and updates placeholder text (tab-switch.js).
Response tab (button#responseTab)	Switches the left textarea to show the stored response JSON (<code>dataStore.responseData`</code>) (tab-switch.js).
JSON textarea (textarea#inputJson, drag/drop enabled)	Accepts typed or dropped <code>.json`</code> /text file content as the request body/source JSON for Pretty, Analyze, and Model actions (input-drop.js); updates <code>dataStore.requestData`</code> while the Request tab is active (tab-switch.js).
Pretty button (button#prettyBtn)	Client-side only — sanitizes and <code>JSON.parse`/JSON.stringify`</code> s the textarea content with 4-space indentation into the output box, or shows an "Invalid JSON" message (make-pretty.js).
Analyze button (button#analyzeBtn)	Parses the textarea JSON and POSTs it to Flask route <code>/analyze`</code> (analyze.py <code>process_json`</code>), which builds an anytree structure and returns a rendered text tree displayed in the output box (validate-json.js).
Model button (button#modelBtn)	Parses the textarea JSON and POSTs it to Flask route <code>/model?language=<selected>`</code> (model.py <code>generate_classes`</code>), which dispatches to the matching generator in <code>web/generators/`</code> and returns generated class/struct source shown in the output box (model-json.js).
Language select	Chooses which backend generator (csharp/python/javascript/cplusplus/java/go) is used when Model is clicked; passed as the <code>language`</code> query parameter to <code>/model`</code> .

(select#languageSelect)

Copy button (button#copyBtn)

Copies the output textarea's text to the clipboard via the Clipboard API, flashing green on success or red on failure/empty content (copy-output.js).

Theme toggle (button#themeToggleBtn)

Swaps the page stylesheet between `style.css` (dark, default) and `lightstyle.css` (light) and updates the icon between sun/moon (theme-toggle.js).